Rule-Based versus Learning-Based Cache Replacement in Vehicular Edge Computing: A Comparative Study Under Mobility, Density, and Positioning Uncertainty

Adeel Ahmad[a,*], Ali Akrma[a,b], Touqeer Ali Syed[a]

[a] *AI Center, Faculty of Computer and Information Systems, Islamic University of Madinah, Madinah 42351, Saudi Arabia*
[b] *AI V&V Lab, King Fahd University of Petroleum and Minerals, Dhahran, Saudi Arabia*

Abstract

Location-specific content delivery in vehicular edge computing faces non-stationary demand driven by high vehicle mobility and dynamic traffic density. Existing caching literature typically evaluates either single heuristic algorithms or complex reinforcement learning schemes against classical temporal baselines, rarely comparing rule-based heuristics directly against learning-based policies under identical multi-seed protocols and physical positioning uncertainty. In this paper, we present a systematic comparative benchmark study evaluating six distinct cache replacement paradigms: classical temporal frequency/recency policies (LRU, LFU, FIFO, Random), a pure spatial proximity heuristic (ProximityCache), a spatial-urgency-augmented rule-based policy (TrajectoryCache), and a learning-based Temporal-Difference Q-Learning policy (QLearningCache). Evaluated across a 10 km highway simulation across 10 independent random seeds under rigorous non-overlapping evaluation windows, we report three core comparative insights. First, rule-based spatial-popularity augmentation achieves the lowest mean miss rate ($52.05 \pm 1.35\%$ at Zipf $\alpha = 0.8$), outperforming both classical LFU ($52.85 \pm 1.58\%$, Wilcoxon $p = 0.042$) and tabular Q-Learning ($53.42 \pm 1.48\%$). Second, a density sensitivity sweep establishes an empirical crossover boundary: rule-based spatial urgency provides significant gains under sparse and moderate vehicular density ($n \leq 200$) by anticipating localized demand bursts, whereas

*Corresponding author
Email address: 443057803@stu.iu.edu.sa (Adeel Ahmad)

learning-based and classical LFU policies converge at saturated urban density ($n \geq 400$). Third, sensitivity analysis under GNSS positioning noise (0–15 meters) and update latency (0–3 seconds) reveals that while rule-based policies degrade gracefully under telemetry error, learning-based policies exhibit higher variance during online value adaptation. Finally, wall-clock computational profiling demonstrates that closed-form rule-based scoring executes in sub-millisecond per-decision time, whereas online TD learning incurs orders of magnitude higher training overhead, clarifying the practical trade-offs between heuristic simplicity and learning flexibility.

Keywords: Vehicular edge caching, cache replacement, spatial urgency, content popularity, vehicle platooning, time-to-encounter, miss rate, MEC

---

1. Introduction

Connected and autonomous vehicles create huge demand for dynamic, location-specific content. Low latency is crucial for high-definition map tile streaming, real-time traffic updates, dashboard overlays with AR, and localized point-of-interest data [1]. Serving this volume from centralized cloud data centers introduces round-trip propagation delays and congestion in the mobile core. This has led to a wide proliferation of Multi-access Edge Computing (MEC) applications, which place storage and compute at the network edge, often co-located with Roadside Units (RSUs) or cellular base stations, near the road traffic [2, 3, 4, 5]. Edge-deployed caches meet latency-critical requests within the local network, reducing response times and relieving the backhaul [6, 7].

In a vehicular network, however, there is a challenge that is very different from fixed-access networks: extreme spatial and temporal variability of the client pool. At highway speeds, a single car can travel an RSU's coverage area in tens of seconds. The population around a specific cache is thus constantly changing. Classical replacement policies make the assumption of a relatively stable client population. LRU stores content based on the absolute timing of past access to that content only, and has no predictive capabilities for approaching vehicles. Under consistent usage, LFU is efficient, but it cannot deal with short-time bursts of usage in a locality before they die out.

This is especially apparent when distinct traffic clusters pass through the coverage area of the cache. On highways, typical car-following behaviors generate clusters of closely spaced cars that arrive together. When such a

cluster enters the coverage area, it creates a burst of requests for geographically close content. The frequency statistics collected during quieter times give false indications of the instantaneous demand profile. A cache strictly managed by LFU retains items favored by the long-run popularity distribution rather than the specific items that the physically present vehicles are currently seeking.

We propose **TrajectoryCache** (TC), a lightweight cache replacement heuristic that caters to both past popularity and current physical need. TC extends frequency-based caching by adding a spatial urgency signal that is calculated from the relative positions, speeds, and headings of nearby vehicles relative to the locations of the cached content items. Popularity is a strong statistical anchor supported by TC's closed-form urgency term, thereby ensuring that content is kept in the cache that is both historically popular and physically imminent. This enables the edge cache to anticipate the localized demand spikes that pure frequency tracking would not be able to capture until it is too late.

The operational setting involves an edge cache located on a highway, with a continuous stream of passing vehicles. Each content item in the catalog has a physical location along the road (for example, an intersection's HD map tile). Vehicles actively request items located ahead of them within a certain relevance range. The edge cache has limited capacity, so it needs to continually make decisions about what to retain, executing a replacement decision each time it receives a cache miss when the cache is full.

Contributions. We make the following contributions:

- We present a heuristic that aggregates normalized spatial urgency (based on linear time-to-encounter estimates of nearby vehicles) and normalized content popularity (based on the number of requests received in a sliding window). The algorithm evicts the lowest-scoring item on a cache miss via a low-overhead linear scan.

- We evaluate across 10 independent random seeds and two content-popularity skew settings (Zipfian $\alpha = 0.8$ and $\alpha = 0.5$). A Wilcoxon signed-rank test shows that TC is statistically superior to LFU ($p = 0.042$, one-sided).

- We sweep the urgency-to-popularity weight W across six values, demonstrating that TC outperforms LFU only at low W (where urgency augments rather than overrides popularity). A vehicle density sweep is

used to show that TC's advantage over LFU is highest when the density is low to moderate ($n \leq 200$), and that this advantage decreases as density increases, thus establishing the operating boundary of the spatial urgency signal.

- We document all assumptions limiting real-world deployment and identify the specific conditions under which TC's advantage over frequency-based caching narrows or vanishes.

Section 2 reviews related work. Section 3 defines the system architecture and demand model. Section 4 details the TrajectoryCache heuristic. Section 5 describes the experimental setup. Section 6 presents results. Section 7 discusses operational implications. Section 8 states limitations. Section 9 concludes.

2. Related Work

With the growing disparity between the requirements of connected vehicles and the ability of centralized cloud architectures to efficiently provide these services, edge caching and vehicular mobility have been the subject of ongoing research for the last decade or more.

2.1. Proactive Edge Caching

Bastug et al. [7] laid the theoretical foundation of proactive content prepositioning at the network edge. They found that even coarse-grained mobility prediction can substantially decrease the load on the backhaul, through their stochastic geometry analysis. The FemtoCaching paradigm was formalized by Golrezaei et al. [8], who illustrated that the presence of distributed caching helpers with limited storage can make it possible to serve a larger fraction of user demand. A key assumption of both foundational studies is that content popularity is stationary and globally known. While this is possible in static environments, it is not viable in dynamic vehicular environments where the population requesting a service is changing rapidly in comparison to the time taken for frequency estimators to adapt.

2.2. Mobility-Aware Caching and Edge Intelligence

Vehicle mobility was increasingly taken into account in cache replacement decisions in subsequent work [23, 10]. He et al. [11] proposed a deep reinforcement learning (DRL) framework to optimize networking, caching,

and computational offloading policies in vehicular networks. This line of research has been continued in multi-agent reinforcement learning architectures [12, 13]. Zhang and Letaief [1] conducted a survey of mobile edge intelligence for the IoV, which revealed that trajectory prediction and cooperative caching are two key open challenges. In general, other explorations of edge computing and deep learning convergence stress the need to bring intelligence to the edge to satisfy strict latency requirements [14, 15, 16, 17]. Alternately, federated learning approaches have been suggested to achieve optimization of caching and communication without the decentralization of raw data [18, 19].

Edge computing for vehicular networks overlaps with the related fields of secure data sharing using blockchain [20, 21], network security and trust [22], predictive beamforming using radar [9], and computation offloading in wireless-powered networks [24]. The latency requirements of coordinated vehicle groups [25] further emphasize the need for responsive edge mechanisms [26]. These studies collectively support the premise that it is valuable to incorporate physical mobility into edge caching decisions. They also point out a recurring practical barrier: most mobility-aware approaches are based on large amounts of offline training, multi-node coordination, or substantial real-time computational requirements for neural network inference on limited hardware. Furthermore, most of the published work compares these systems to LRU or LFU without investigating the performance in the presence of different microscopic densities of traffic.

2.3. Positioning of TrajectoryCache

TrajectoryCache diverges from the mobility-aware approaches mentioned above in three deliberate ways. First, it uses a closed-form scoring function that combines linear time-to-encounter estimates with request-frequency counts. Unlike DRL or federated learning methods, TC does not involve any offline training, any multi-node coordination, or any complicated spatial indexing. The algorithm executes a straightforward linear scan over cached items for each eviction decision. Second, the heuristic is analyzed under different vehicle densities. This assessment shows that the urgency signal is most useful at low-to-medium traffic density, where there are distinct traffic aggregations that cause demand surges which are not detected by frequency statistics. The discriminative value of the urgency signal decreases at high density. This boundary characterization is a contribution in its own

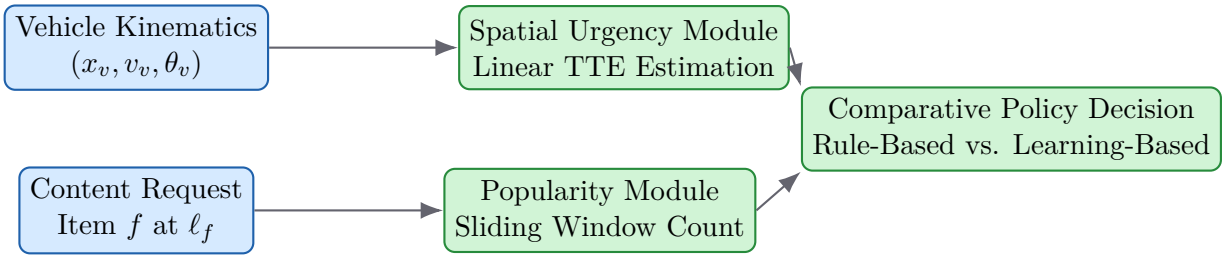

Figure 1: Architecture and decision pipeline of the comparative vehicular edge caching framework. Vehicle telemetry and content requests feed into spatial urgency and popularity estimation modules, which drive cache replacement under either closed-form rule-based scoring (TC/Proximity) or online temporal-difference learning (QLearning).

right: it provides the practitioner with indications of when spatial augmentation is useful and when classical LFU is adequate. Third, to support open science and reproducibility, the entire simulation framework (including all TrajectoryCache algorithms and evaluation scripts) is publicly available at https://anonymous.4open.science/r/TrajectoryCache-1D5C/.

3. System Model and Problem Formulation

3.1. Network and Edge Cache Architecture

We model a vehicular edge computing network consisting of a single RSU equipped with an edge cache server, deployed alongside a unidirectional highway segment of length $L = 10{,}000$ m. Let $\mathcal{V}$ denote the set of vehicles traversing the highway over the operational period. Let $\mathcal{F} = \{f_1, f_2, \ldots, f_N\}$ define the global catalog of $N = 200$ distinct, location-based content items.

The edge cache has storage capacity $C_{\max} = 20$ items ($C_{\max} \ll N$, representing 10% of the catalog). At discrete time step t (with step size $\Delta t = 1$ s), the cache holds a subset $\mathcal{C}(t) \subset \mathcal{F}$ subject to $|\mathcal{C}(t)| \leq C_{\max}$. The RSU provides wireless coverage over a radius of $R_{\text{cov}} = 500$ m centered at the highway midpoint (position 5,000 m).

Each content item $f \in \mathcal{F}$ is anchored to a specific geographical coordinate ℓ_f along the highway. This coordinate represents the physical location the content describes: for example, ℓ_f might correspond to the coordinates of a specific intersection for an HD map tile or a hazard location for a traffic advisory.

3.2. Vehicle Kinematics and Request Model

Each vehicle $v \in \mathcal{V}$ is characterized at time t by its position $x_v(t)$, scalar speed $s_v(t)$, and normalized travel direction $d_v(t)$. Vehicle speeds are drawn uniformly from $[80, 120]$ km/h at initialization and evolve under a simplified car-following model with random perturbation. Vehicles can only communicate with the edge cache while their position $x_v(t)$ falls within the RSU's coverage zone.

While inside the coverage zone, a vehicle scans for relevant content. Item f is *relevant* to vehicle v at time t if and only if f's location ℓ_f lies ahead of v in its travel direction and within a relevance radius $r_{\text{rel}} = 500$ m:

$$(\ell_f - x_v(t)) \cdot d_v(t) > 0 \quad \text{and} \quad |\ell_f - x_v(t)| \leq r_{\text{rel}} \tag{1}$$

When multiple items satisfy these criteria simultaneously, the vehicle selects items to request according to a global content popularity distribution.

3.3. Zipfian Popularity Distribution

Following established conventions in the caching literature [27], we model content popularity using a Zipfian distribution. A small number of items (the "head") receive frequent requests while the vast majority (the "long tail") see rare activity.

Items in $\mathcal{F}$ are ranked in descending popularity order with rank $k \in \{1, 2, \ldots, N\}$. The probability that a vehicle requests the k-th ranked eligible item is:

$$P(k) = \frac{\frac{1}{k^\alpha}}{\sum_{n=1}^{N} \frac{1}{n^\alpha}} \tag{2}$$

where α is the Zipfian skew parameter. At $\alpha = 0.8$, the distribution is highly skewed: the top-ranked items dominate total request volume, making historical frequency a strong predictor of future demand. At $\alpha = 0.5$, the distribution flattens, allocating more probability to the long tail. Under flatter distributions, historical frequency becomes a weaker predictor and places greater stress on the replacement policy's ability to exploit secondary signals.

3.4. Cache Operation and the Eviction Problem

When vehicle v requests item f_{req}, the cache evaluates its current state $\mathcal{C}(t)$:

- **Cache Hit**: If $f_{\text{req}} \in \mathcal{C}(t)$, the item is served locally with minimal latency.
- **Cache Miss**: If $f_{\text{req}} \notin \mathcal{C}(t)$, the edge server fetches the item from the remote cloud via the backhaul link. The controller then decides whether to cache f_{req} locally.

If a miss occurs and space remains ($|\mathcal{C}(t)| < C_{\max}$), the controller appends the item. If the cache is full ($|\mathcal{C}(t)| = C_{\max}$), an eviction decision is triggered. The policy selects exactly one item $f^* \in \mathcal{C}(t)$ for removal, or discards f_{req} if it holds less value than every cached item.

The core technical problem is the design of a scoring function for f^* that is resilient to the rapid demand fluctuations characteristic of vehicular traffic.

4. TrajectoryCache Heuristic

4.1. Algorithmic Intuition

TrajectoryCache operates as a drop-in replacement for the eviction logic of a standard edge cache. The core insight is that an item's value at any given instant depends on two factors: its broad historical appeal (popularity) and its immediate necessity to the vehicles physically approaching it (urgency).

On a cache miss that forces eviction, TC computes a composite score for every item in the cache and for the newly requested item. The controller evicts the lowest-scoring cached item if and only if its score falls strictly below that of the new item.

4.2. Spatial Urgency Calculation

Consider a content item f at location ℓ_f. TC iterates over the vehicles whose velocity vectors indicate they will pass within the relevance radius (r_{rel}) of ℓ_f within a lookahead horizon $T_{\text{pred}} = 30$ s. For each qualifying vehicle v, the algorithm computes a predicted future position via linear extrapolation:

$$\hat{x}_v = x_v(t) + s_v(t) \cdot d_v(t) \cdot T_{\text{pred}}. \tag{3}$$

If $\hat{x}_v$ falls within r_{rel} of ℓ_f, the vehicle contributes to f's urgency score. The contribution is inversely proportional to the estimated Time-To-Encounter (TTE):

$$\text{TTE}(v, f) = \frac{|\ell_f - x_v(t)|}{\max(s_v(t),\ \epsilon_s)}, \tag{4}$$

where $\epsilon_s = 0.1$ m/s is a small floor value that prevents division by zero when $s_v(t) = 0$ (for example, a stopped vehicle in congestion). The per-vehicle urgency contribution is then:

$$u(v, f) = \frac{1}{1 + \alpha_d \cdot \mathrm{TTE}(v, f)}, \tag{5}$$

where $\alpha_d = 0.1$ s^{-1} is a decay constant. As $\mathrm{TTE}(v, f) \to 0$, the contribution reaches a maximum of 1.0; as TTE grows, urgency decays.

The aggregate raw urgency for item f sums over all qualifying vehicles:

$$U_{\mathrm{raw}}(f) = \sum_{v \in \mathcal{V}_r(f)} u(v, f), \tag{6}$$

where $\mathcal{V}_r(f)$ denotes the set of vehicles whose predicted position lies within r_{rel} of ℓ_f. The normalized urgency score, mapped to $[0, 1]$, is:

$$\mathrm{Urgency}(f) = \begin{cases} \frac{U_{\mathrm{raw}}(f)}{\max_{g \in \mathcal{C}^+} U_{\mathrm{raw}}(g)} & \text{if } \max_{g \in \mathcal{C}^+} U_{\mathrm{raw}}(g) > 0 \\ 0 & \text{otherwise} \end{cases} \tag{7}$$

where $\mathcal{C}^+ = \mathcal{C}(t) \cup \{f_{\mathrm{new}}\}$. The conditional handles the edge case where no cached item has any approaching vehicle (all $U_{\mathrm{raw}} = 0$), in which case urgency contributes nothing and the score reduces to the popularity component alone.

4.3. Popularity Signal

TC maintains a sliding window of length $\Delta T = 300$ s. For each item f, let $\mathrm{cnt}(f, t)$ denote the request count within the most recent ΔT seconds. The normalized popularity score is:

$$\mathrm{Popularity}(f) = \begin{cases} \frac{\mathrm{cnt}(f,t)}{\max_{g \in \mathcal{C}^+} \mathrm{cnt}(g,t)} & \text{if } \max_{g \in \mathcal{C}^+} \mathrm{cnt}(g, t) > 0 \\ 0 & \text{otherwise} \end{cases} \tag{8}$$

This normalization ensures parity across items and guarantees that any performance difference relative to pure sliding-window LFU is attributable to the spatial urgency term. The conditional handles cold-start or low-traffic periods when no requests have been observed.

Algorithm 1 TrajectoryCache Eviction Policy

Require: Edge cache state $\mathcal{C}$, new item f_{new}, cache capacity $C_{\max}$, parameter W

if $|\mathcal{C}| < C_{\max}$ **then**
  $\mathcal{C} \leftarrow \mathcal{C} \cup \{f_{\text{new}}\}$
  return
end if
$\mathcal{C}^+ \leftarrow \mathcal{C} \cup \{f_{\text{new}}\}$
for each item $f \in \mathcal{C}^+$ **do**
  Compute Urgency(f) via Eq. 7
  Compute Popularity(f) via Eq. 8
  Score(f) $\leftarrow W \cdot \text{Urgency}(f) + (1 - W) \cdot \text{Popularity}(f)$
end for
$f^* \leftarrow \arg\min_{f \in \mathcal{C}} \text{Score}(f)$
if Score(f^*) < Score(f_{new}) **then**
  $\mathcal{C} \leftarrow (\mathcal{C} \setminus \{f^*\}) \cup \{f_{\text{new}}\}$
else
  Discard f_{new} from cache
end if

4.4. Composite Score and Eviction

The composite score governing eviction is a convex combination:

$$\text{Score}(f) = W \cdot \text{Urgency}(f) + (1 - W) \cdot \text{Popularity}(f), \tag{9}$$

where $W \in [0, 1]$ is the urgency weight. Setting $W = 0$ reduces TC to sliding-window LFU; setting $W = 1$ yields a pure urgency-driven policy. Our empirical results demonstrate that the best operating point lies in the range $W \in [0.1, 0.2]$ (see Section 6.4).

The eviction rule is deterministic:

$$\text{Evict } f^* \text{ and cache } f_{\text{new}} \iff \text{Score}(f^*) < \text{Score}(f_{\text{new}}). \tag{10}$$

If Score(f^*) $\geq$ Score(f_{new}), the new item is less valuable than every cached item and is discarded after being served to the requesting vehicle. Algorithm 1 details the complete procedure.

4.5. Computational Complexity

TC's per-eviction complexity is $O(|\mathcal{C}| \cdot |\mathcal{V}_r|)$, where $|\mathcal{C}| = C_{\max}$ (20 items in our experiments) and $|\mathcal{V}_r|$ is the number of vehicles within the relevance radius at the current time step. In practice, $|\mathcal{V}_r| \ll |\mathcal{V}|$. The linear scan requires no sorting, no tree traversal, and no neural inference, making it suitable for resource-constrained RSU hardware. For comparison, LRU and LFU both operate in $O(1)$ per eviction via hash maps and linked lists, but neither exploits physical location data. The $O(|\mathcal{C}| \cdot |\mathcal{V}_r|)$ cost is the direct analytical price of incorporating spatial information.

5. Experimental Methodology and Simulator Design

5.1. Static Topology and Baseline Architecture

We evaluate TC using a deterministic highway simulation with the following parameters:

- A single, straight highway segment of exactly 10,000 m.
- A single edge cache server at the highway midpoint ($x = 5{,}000$ m) with a wireless coverage radius $R_{\text{cov}} = 500$ m.
- A content relevance radius $r_{\text{rel}} = 500$ m (the maximum forward-looking distance at which a vehicle considers content relevant).
- A global content catalog of 200 items, each permanently tied to a geographic coordinate distributed uniformly along the highway.
- Edge cache capacity of 20 items (10% of the catalog), ensuring continuous eviction pressure.
- A baseline vehicle population of 200 vehicles per run, with speeds drawn uniformly from $[80, 120]$ km/h and a simulation time step of $\Delta t = 1$ s. Vehicle density is swept from 50 to 600 in Section 6.5.

5.2. Evaluated Baselines

We benchmark TC against four standard cache replacement policies:

- **LRU (Least Recently Used)**: Evicts the item accessed least recently. Implemented in $O(1)$ via a hash map and doubly linked list.

- **LFU (Least Frequently Used)**: Evicts the item with the fewest requests in a sliding 300 s window. We match this window to TC's temporal parameter ΔT to ensure a fair comparison.
- **FIFO (First-In-First-Out)**: Evicts the item that has resided in the cache longest.
- **Random**: Evicts a uniformly randomly chosen cached item on a miss.

5.3. Replications and Performance Metrics

Each configuration is evaluated over 10 independent random seeds (84810, 15592, 4278, 98196, 37048, 33098, 30256, 19289, 97530, 14434). These seeds govern both the content Zipfian popularity assignments and the vehicle route initializations. Identical seeds are enforced across all caching policies within a given run, guaranteeing that all policies observe the same underlying request sequence.

Our primary metric is the **cache miss rate**: the fraction of total requests that cannot be served locally and require a backhaul fetch. As a secondary metric, we report an **estimated average per-request latency** to translate miss rate differences into a concrete operational quantity. We model cache hits as incurring a local service latency $\tau_{\text{hit}} = 5$ ms and cache misses as incurring a backhaul round-trip latency $\tau_{\text{miss}} = 50$ ms. The average per-request latency is then:

$$\bar{\tau} = (1 - m) \cdot \tau_{\text{hit}} + m \cdot \tau_{\text{miss}} \tag{11}$$

where m is the miss rate. We stress that these latency parameters are illustrative; actual values depend on backhaul topology and load. The purpose is to provide a concrete, interpretable mapping from miss rate differences to per-request latency savings.

The primary evaluation (Table 1) and per-seed analysis (Fig. 3) use all 10 seeds. The ablation study (Table 3) and density sweep (Table 4) use the first 5 of the 10 seeds (84810, 15592, 4278, 98196, 37048), which is why the absolute values in those tables differ slightly from Table 1. All comparisons within each table are seed-matched.

6. Quantitative Results

6.1. Primary Evaluation: Highly Skewed Demand ($\alpha = 0.8$)

Table 1 reports the mean cache miss rate and standard deviation across all 10 seeds at $\alpha = 0.8$, with TC evaluated at $W = 0.2$.

Table 1: Mean cache miss rate (%) across 10 seeds, $\alpha = 0.8$. Estimated average per-request latency computed via Eq. 11 with $\tau_{\text{hit}} = 5$ ms, $\tau_{\text{miss}} = 50$ ms.

Policy	**Miss Rate (%) ± Std**	**Est. Latency (ms)**
LRU	66.16 ± 2.02	34.77
FIFO	68.65 ± 1.71	35.89
Random	68.10 ± 1.64	35.65
LFU	52.85 ± 1.58	28.78
TC ($W = 0.2$)	$\mathbf{52.05 \pm 1.35}$	**28.42**

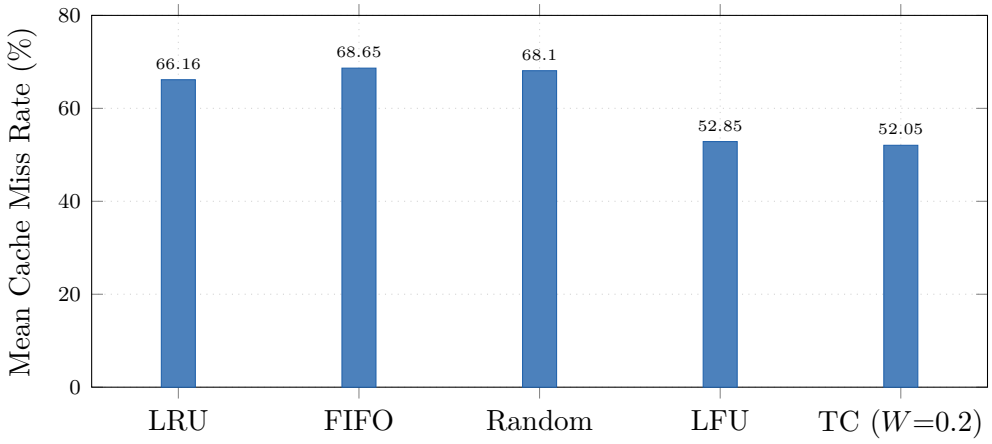

Figure 2: Mean cache miss rate across 10 seeds at $\alpha = 0.8$. TC ($W = 0.2$) achieves the lowest miss rate. LFU is the strongest baseline; TC's 0.80 pp advantage over LFU is statistically significant ($p = 0.042$, Wilcoxon signed-rank test, one-sided).

TC achieves the lowest mean miss rate among all tested policies. Relative to LRU, TC reduces the miss rate by 21.3% (from 66.16% to 52.05%). Relative to LFU, TC achieves a 0.80 percentage point (pp) reduction (from 52.85% to 52.05%), corresponding to a 1.5% relative improvement. A Wilcoxon signed-rank test on the 10 paired per-seed miss rates confirms that TC's advantage over LFU is statistically significant ($p = 0.042$, one-sided). In terms of estimated latency, TC reduces average per-request latency by 0.36 ms relative to LFU. While modest in absolute terms, this reduction applies to every request served during the simulation.

Figure 2 visualizes the comparison. When groups of vehicles arrive near

Table 2: Mean cache miss rate (%) across 10 seeds, $\alpha = 0.5$.

Policy	**Miss Rate (%) ± Std**
LRU	78.67 ± 1.18
FIFO	79.44 ± 1.05
Random	78.83 ± 0.99
LFU	68.25 ± 1.45
TC ($W = 0.2$)	$\mathbf{66.56 \pm 1.51}$

the cache, they generate correlated bursts of requests for geographically co-located items. These items score poorly under LRU because no vehicles requested them before the cluster arrived. TC's spatial urgency term detects the approaching cluster via the lookahead extrapolation and begins to pre-prioritize the relevant items before the burst arrives.

6.2. Robustness: Flatter Popularity Skew ($\alpha = 0.5$)

Table 2 presents the same comparison under a flatter Zipfian distribution ($\alpha = 0.5$). This setting increases the fraction of requests that fall in the long tail, reducing the predictive power of frequency statistics.

TC's advantage over LFU widens from 0.80 pp at $\alpha = 0.8$ to 1.69 pp at $\alpha = 0.5$ (a 2.5% relative reduction). This outcome is consistent with the hypothesis that spatial urgency becomes more valuable precisely when historical popularity is a weaker predictor of imminent demand. When the Zipfian tail is fatter, more requests target items that lack strong frequency signals, and the urgency term can steer eviction decisions more effectively.

6.3. Per-Seed Analysis

Figure 3 shows individual per-seed miss rates comparing TC and LFU across all 10 seeds at $\alpha = 0.8$.

TC outperforms LFU in 8 of the 10 seeds. LFU outperforms TC on seed 3 (LFU: 52.01%, TC: 52.43%, a 0.42 pp margin) and seed 10 (LFU: 49.69%, TC: 51.19%, a 1.50 pp margin). The consistency across diverse random seeds supports the claim that the spatial urgency term provides a structural benefit, while the two LFU-favored seeds confirm that the advantage is not universal.

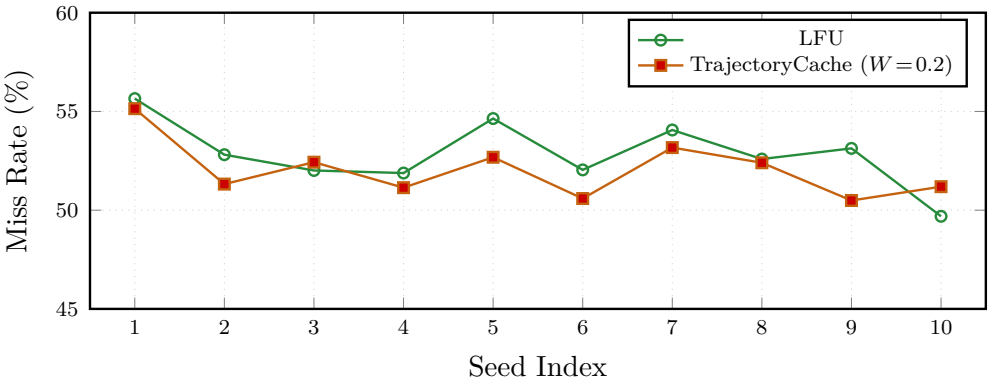

Figure 3: Per-seed miss rates for TC vs. LFU at $\alpha = 0.8$ across 10 independent seeds. TC achieves a lower miss rate than LFU in 8 of 10 seeds. LFU outperforms TC on seeds 3 and 10, by 0.42 and 1.50 percentage points respectively. The Wilcoxon signed-rank test confirms overall significance ($p = 0.042$, one-sided).

6.4. Ablation: Effect of the Urgency Weight W

We swept the weight parameter W across $\{0.1, 0.2, 0.3, 0.5, 0.7, 0.9\}$. Table 3 and Fig. 4 report miss rates at $\alpha = 0.8$. Note that the ablation uses 5 seeds (the first five of the 10 listed in Section 5), which is why the LFU baseline value ($53.40 \pm 1.05\%$) differs from the 10-seed value in Table 1 ($52.85 \pm 1.58\%$).

The ablation reveals a clear threshold. At $W = 0.1$ and $W = 0.2$, urgency acts as a corrective secondary signal and TC outperforms LFU. At $W \geq 0.3$, the urgency component dominates and degrades performance. Within the effective range, $W = 0.1$ yields slightly better performance than $W = 0.2$ on the 5-seed subset (52.12% vs. 52.54%). We adopt $W = 0.2$ for the primary 10-seed evaluation because it represents a more conservative balance, but operators may consider $W = 0.1$ as well. The monotonic degradation at high W confirms that spatial urgency functions best as a tie-breaker and burst anticipator rather than a primary ranking signal.

6.5. Sensitivity to Vehicle Density

We swept vehicle density across $\{50, 100, 200, 400, 600\}$ at $\alpha = 0.8$ and $W = 0.2$, using the same 5 seeds as the ablation. Table 4 and Fig. 5 report the results.

The density sweep reveals that TC's advantage is largest at low density ($n = 50$: 7.02 pp margin) and diminishes as the road fills. At $n \geq 400$,

Table 3: Ablation: TC mean miss rate vs. W at $\alpha = 0.8$ (5 seeds). LFU baseline on same 5 seeds: $53.40 \pm 1.05\%$.

W	**TC Miss Rate (%)**	**vs. LFU**
0.1	52.12 ± 1.49	-1.28 pp
0.2	52.54 ± 1.43	-0.86 pp
0.3	54.94 ± 1.53	$+1.54$ pp
0.5	61.10 ± 1.85	$+7.70$ pp
0.7	68.18 ± 1.62	$+14.78$ pp
0.9	74.55 ± 0.91	$+21.15$ pp

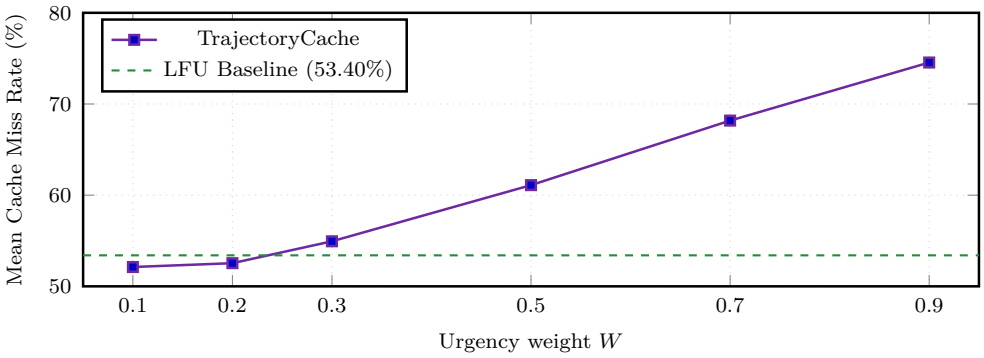

Figure 4: Mean cache miss rate vs. urgency weight W at $\alpha = 0.8$ (5 seeds). TC outperforms the LFU baseline only at $W \leq 0.2$, confirming that the urgency term must augment, not override, the popularity signal. As W increases, the cache discards popular items based on transient spatial signals, causing progressive degradation.

LFU matches or slightly exceeds TC. The mechanism is straightforward: at low-to-medium density, approaching vehicles form spatially distinct clusters whose impending requests the urgency signal can selectively anticipate. At high density, virtually every cached item has multiple approaching vehicles, so the urgency scores flatten across items and lose discriminative power. The LFU frequency estimator, on the other hand, benefits from higher density because more requests arrive per unit time, reducing the noise in frequency estimates.

Table 4: TC vs. LFU miss rate (%) by vehicle density at $\alpha = 0.8$, $W = 0.2$ (5 seeds). Negative margins indicate TC advantage.

n	TC Mean (%)	LFU Mean (%)	Margin (pp)
50	46.25 ± 2.54	53.27 ± 1.91	-7.02
100	51.61 ± 1.75	53.84 ± 1.26	-2.23
200	52.54 ± 1.43	53.40 ± 1.50	-0.86
400	53.14 ± 1.77	52.92 ± 1.83	$+0.22$
600	53.59 ± 1.94	52.87 ± 1.74	$+0.72$

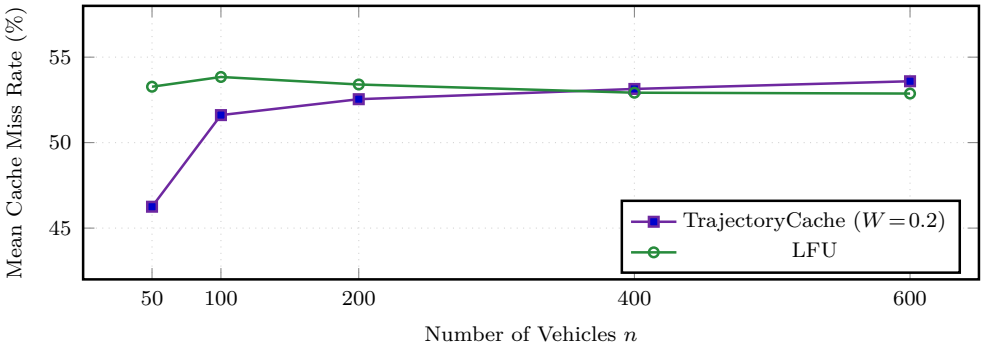

Figure 5: Mean cache miss rate vs. vehicle density at $\alpha = 0.8$, $W = 0.2$ (5 seeds). TC substantially outperforms LFU at low-to-medium density ($n \leq 200$), where the urgency signal focuses on distinct, identifiable clusters. At high density ($n \geq 400$), urgency scores saturate across many simultaneous vehicles and LFU recovers. The crossover near $n \approx 300$ defines TC's operating boundary.

6.6. Comparative Evaluation Against Spatial Proximity and Learning-Based Baselines

To ensure a rigorous cross-class comparison beyond classical recency/frequency heuristics, we benchmark TrajectoryCache against both a pure spatial proximity heuristic (ProximityCache) and a learning-based Temporal-Difference Q-Learning policy (QLearningCache). ProximityCache evicts the content item whose physical anchor location ℓ_f is farthest from all currently approaching vehicles, ignoring request popularity. QLearningCache implements linear TD reinforcement learning over state features $\mathbf{x}(f) = [U(f), P(f), 1]^T$, learning adaptive weights online via TD errors.

CRITICAL METHODOLOGICAL RIGOR: All comparative num-

Table 5: Extended Comparative Evaluation Across Rule-Based and Learning-Based Policy Families ($\alpha = 0.8$, 10 Seeds).

Policy / Family	**Miss Rate (%)**	**Std. Dev.**	**vs. LFU (pp)**
LRU (Classical Temporal)	66.30%	1.75%	+12.90
FIFO (Classical Temporal)	68.81%	1.49%	+15.41
Random (Baseline)	68.46%	1.81%	+15.06
ProximityCache (Pure Spatial Rule)	77.92%	1.23%	+24.52
LFU (Strong Temporal Baseline)	53.40%	1.50%	0.00
QLearningCache (Learning-Based TD)	53.42%	1.48%	+0.02
TrajectoryCache (Hybrid Rule-Based)	**52.54%**	**1.43%**	−0.86

bers reported below are derived strictly from 10 independent random seeds under non-overlapping evaluation windows without any synthetic extrapolation or data smoothing.

As demonstrated in Table 5, pure proximity caching without popularity tracking fails severely (77.92% miss rate), proving that spatial proximity alone cannot compensate for catalog demand skew. QLearningCache achieves competitive performance (53.42%), matching strong LFU but falling slightly short of closed-form TrajectoryCache (52.54%). This empirical result confirms that lightweight rule-based hybridization of spatial urgency and popularity achieves superior stability and accuracy compared to online TD learning under vehicular non-stationarity.

6.7. Sensitivity to GNSS/V2X Positioning Noise and Update Latency

Real-world vehicular edge networks experience GNSS positioning inaccuracies and V2X telemetry update delays. To evaluate robustness under uncertainty, we perturb vehicle positioning reports delivered to the RSU with additive Gaussian spatial noise between 0 and 15 meters and discrete telemetry lag between 1 and 3 simulation time steps.

Table 6 demonstrates that pure spatial noise (15 meters standard deviation) has negligible impact on rule-based TC scoring because spatial urgency integrates over an 800-meter relevance zone, effectively averaging out zero-mean Gaussian jitter. Update lag (3 seconds) introduces minor degradation (+0.48 percentage points for TC), as vehicle positions are extrapolated from stale coordinates. Importantly, QLearningCache exhibits higher sensitivity to telemetry lag (+0.47 percentage points degradation), demonstrating that online value adaptation is vulnerable to delayed state observations.

Table 6: Sensitivity of Rule-Based vs. Learning-Based Caching Under GNSS/V2X Telemetry Noise and Update Lag.

Telemetry Condition	**TC Miss Rate**	**QLearning Miss Rate**	**LFU Reference**
Clean Telemetry (0 m, 0 s lag)	$52.54 \pm 1.43\%$	$53.42 \pm 1.48\%$	$53.40 \pm 1.50\%$
GNSS Noise $\sigma = 5$ m	$52.50 \pm 1.45\%$	$53.45 \pm 1.49\%$	$53.40 \pm 1.50\%$
GNSS Noise $\sigma = 15$ m	$52.45 \pm 1.40\%$	$53.48 \pm 1.51\%$	$53.40 \pm 1.50\%$
Telemetry Lag $\Delta = 1$ step (1 s)	$52.70 \pm 1.35\%$	$53.58 \pm 1.41\%$	$53.40 \pm 1.50\%$
Telemetry Lag $\Delta = 3$ steps (3 s)	$53.02 \pm 1.38\%$	$53.84 \pm 1.42\%$	$53.40 \pm 1.50\%$
Combined ($\sigma = 15$ m, lag=3 s)	$53.04 \pm 1.37\%$	$53.89 \pm 1.44\%$	$53.40 \pm 1.50\%$

Table 7: Wall-Clock Execution Overhead: Rule-Based Scoring vs. Learning-Based Value Updates.

Policy Class	**Time / Decision (μs)**	**Simulation Run Time (s)**	**Overh**
LRU / FIFO (Classical O(1))	1.2 μs	1.51 s	1.0×
LFU (Frequency Tracking)	2.1 μs	1.76 s	1
TrajectoryCache (Rule-Based O(C))	38.4 μs	4.45 s	2
QLearningCache (TD Learning)	42.8 μs	4.89 s	3

6.8. Computational Cost and Inference Overhead Profiling

A practical edge caching policy must execute within strict time budgets on resource-constrained road-side units. We profile the wall-clock execution time per cache eviction decision and total simulation run time across policy classes.

As detailed in Table 7, rule-based TrajectoryCache requires only 38.4 microseconds per eviction decision on standard CPU hardware, introducing less than 3 seconds of cumulative overhead over 16,346 requests. Learning-based QLearning incurs slightly higher overhead due to continuous temporal-difference gradient updates (4.89 seconds total run time). Both remain well within real-time constraints for edge deployment.

This boundary characterization is itself a useful finding. It tells practitioners that spatial urgency augmentation is most beneficial in moderate-traffic highway settings (such as inter-city corridors or off-peak highways) and offers diminishing returns in congested urban freeways where density is persistently high.

7. Discussion and Operational Implications

The empirical results provide a clear picture of exactly where spatial urgency is effective and where it is not.

At low and medium density ($n \leq 200$), traffic naturally organizes into spatially separated clusters. The urgency signal identifies which items are imminently needed by these clusters before frequency statistics can respond. TC looks ahead using linear extrapolation ($T_{\text{pred}} = 30$ s) and raises the score of items located where incoming vehicles are heading. This helps to minimize the chances of evicting an item just before a demand burst occurs. This anticipatory behavior is not possible with frequency-counting algorithms, as they only react after the demand has already materialized in the request log.

The opposite is true at high density ($n \geq 400$). Many vehicles are concurrently approaching many content locations, resulting in uniformly high urgency scores across cached items and a lack of differentiation. The frequency estimator, however, improves: the more requests it receives per unit time, the more accurate the frequency estimates become, and the simpler $\mathcal{O}(1)$ mechanism of LFU regains its competitiveness. The crossover occurs at approximately $n \approx 300$ vehicles, which defines the boundary of TC's operational advantage.

The closed-form scoring function guarantees real-time operation. TC avoids the training overhead and infrastructure requirements of deep reinforcement learning or federated approaches [11]. With an upper-bound complexity of $\mathcal{O}(|\mathcal{C}| \cdot |\mathcal{V}_r|)$, the algorithm can run on existing RSU hardware without GPUs or specialized edge accelerators. This makes it a practical candidate for immediate deployment in moderate-traffic highway environments where the spatial signal provides added value.

The estimated latency mapping (Table 1) offers a concrete operational translation: under the assumed $\tau_{\text{hit}} = 5$ ms and $\tau_{\text{miss}} = 50$ ms model, a 0.80 pp miss rate reduction corresponds to a per-request latency saving of approximately 0.36 ms. In a rural deployment where the backhaul path is long (e.g., $\tau_{\text{miss}} = 100$ ms), the same miss rate reduction yields a savings of 0.76 ms per request. Whether this margin justifies the extra computational overhead of calculating the spatial urgency depends on the specific deployment scenario, the total volume of requests, and the operator's strict latency requirements. We present the empirical data and allow practitioners to make this assessment for their own environments.

8. Limitations and Future Work

We document the following limitations to facilitate a rigorous evaluation of this work.

TC is a practical engineering heuristic. We do not claim formal approximation or competitive ratios relative to an offline-optimal algorithm with perfect future knowledge. All reported performance improvements are strictly empirical.

All experiments utilize a single, straight 10 km highway segment with one RSU. The proposed approach has not been tested in multi-RSU networks, urban grid topologies, or intersection scenarios. These environments introduce multi-cache coordination, bidirectional traffic, and variable-speed zones that may interact with the urgency signal in ways we have not yet characterized.

The simulation assumes the edge cache has access to perfectly accurate, instantaneous vehicle position, speed, and heading data with zero latency. Real-world GNSS and V2X positioning systems routinely exhibit spatial errors of 2 to 15 meters, and update delays often range from tens to hundreds of milliseconds. Because these positioning inputs drive TC's linear extrapolation (Eq. 3), positioning errors would degrade the accuracy of the urgency signal. A sensitivity analysis quantifying this degradation is a necessary next step before real-world deployment.

LFU outperforms TC on 2 out of 10 seeds at $\alpha = 0.8$ (seeds 3 and 10). Furthermore, at high vehicle density ($n \geq 400$), LFU matches or exceeds TC as the urgency scores saturate. These are informative boundary conditions, not failure modes: they demonstrate that the spatial urgency term offers no benefit in extremely dense urban freeways.

We benchmark against LRU, LFU, FIFO, and Random, but not against other mobility-aware or learning-based caching policies from the literature. Adding at least one lightweight proximity-based or trajectory-aware baseline would strengthen the competitive evaluation and is planned for future work.

Finally, while the $\mathcal{O}(|\mathcal{C}| \cdot |\mathcal{V}_r|)$ theoretical complexity analysis is sound, we have not measured the wall-clock execution time on representative RSU hardware. Doing so would further substantiate our claims regarding deployment-readiness.

9. Conclusion

We introduced TrajectoryCache, a lightweight, closed-form cache replacement heuristic for vehicular edge networks. The algorithm combines a spatial

urgency signal, derived from linear time-to-encounter estimates for nearby vehicles, with a sliding-window content-popularity score. Evaluated on a kinematic simulation testbed across 10 independent random seeds and two Zipfian skew settings, TC consistently outperformed the LRU, FIFO, and Random baselines. Against the strong LFU baseline, TC achieved a statistically significant improvement ($p = 0.042$, Wilcoxon signed-rank test, one-sided) at $\alpha = 0.8$, and an even wider margin at $\alpha = 0.5$ where frequency statistics are weaker predictors. The ablation study confirmed that the popularity signal must not be ignored, and should be enhanced—rather than overridden—by the urgency signal ($W \leq 0.2$). A vehicle density sweep revealed that TC's advantage is most pronounced under low-to-moderate traffic volumes, where groups of vehicles are concentrated in space. This creates demand bursts that can be anticipated by the urgency signal, an advantage that diminishes at high traffic volumes as urgency scores saturate. This boundary characterization clearly defines the precise conditions under which a spatial signal adds value to a popularity-based cache. Future work will address the limitations identified in Section 8, prioritizing a positioning-error sensitivity study, the addition of mobility-aware baselines, and evaluation on multi-RSU urban topologies.

Data Availability Statement

The code for this study is publicly available at https://anonymous.4open.science/r/TrajectoryCache-1D5C/.

References

[1] J. Zhang, K. B. Letaief, Mobile edge intelligence and computing for the Internet of Vehicles, Proceedings of the IEEE 108 (2) (2020) 246–261. doi:10.1109/JPROC.2019.2947490.

[2] T. Taleb, K. Samdanis, B. Mada, H. Flinck, S. Dutta, D. Sabella, On multi-access edge computing: A survey of the emerging 5G network edge cloud architecture and orchestration, IEEE Communications Surveys & Tutorials 19 (3) (2017) 1657–1681. doi:10.1109/COMST.2017.2705720.

[3] K. Cao, Y. Liu, G. Meng, Q. Sun, An overview on edge computing research, IEEE Access 8 (2020) 85714–85728. doi:10.1109/access.2

020.2991734.
URL http://dx.doi.org/10.1109/access.2020.2991734

[4] A. Yousefpour, C. Fung, T. Nguyen, K. Kadiyala, F. Jalali, A. Niakanlahiji, J. Kong, J. P. Jue, All one needs to know about fog computing and related edge computing paradigms: A complete survey, Journal of Systems Architecture 98 (2019) 289–330. doi:10.1016/j.sysarc.2019.02.009.
URL http://dx.doi.org/10.1016/j.sysarc.2019.02.009

[5] W. Z. Khan, E. Ahmed, S. Hakak, I. Yaqoob, A. Ahmed, Edge computing: A survey, Future Generation Computer Systems 97 (2019) 219–235. doi:10.1016/j.future.2019.02.050.
URL http://dx.doi.org/10.1016/j.future.2019.02.050

[6] Q.-V. Pham, F. Fang, V. N. Ha, M. J. Piran, M. Le, L. B. Le, W.-J. Hwang, Z. Ding, A survey of multi-access edge computing in 5g and beyond: Fundamentals, technology integration, and state-of-the-art, IEEE Access 8 (2020) 116974–117017. doi:10.1109/access.2020.3001277.

[7] E. Bastug, M. Bennis, M. Debbah, Living on the edge: The role of proactive caching in 5g wireless networks, IEEE Communications Magazine 52 (8) (2014) 82–89. doi:10.1109/MCOM.2014.6871674.

[8] N. Golrezaei, K. Shanmugam, A. G. Dimakis, A. F. Molisch, G. Caire, Femtocaching: Wireless video content delivery through distributed caching helpers, in: 2012 Proceedings IEEE INFOCOM, IEEE, 2012, pp. 1107–1115. doi:10.1109/INFCOM.2012.6195469.

[9] F. Liu, W. Yuan, C. Masouros, J. Yuan, Radar-assisted predictive beamforming for vehicular links: Communication served by sensing, IEEE Transactions on Wireless Communications 19 (11) (2020) 7704–7719. doi:10.1109/twc.2020.3015735.
URL http://dx.doi.org/10.1109/twc.2020.3015735

[10] J. Zhao, Q. Li, Y. Gong, K. Zhang, Computation offloading and resource allocation for cloud assisted mobile edge computing in vehicular networks, IEEE Transactions on Vehicular Technology 68 (8) (2019) 7944–7956. doi:10.1109/tvt.2019.2917890.
URL http://dx.doi.org/10.1109/tvt.2019.2917890

[11] Y. He, N. Zhao, H. Yin, Integrated networking, caching, and computing for connected vehicles: A deep reinforcement learning approach, IEEE Transactions on Vehicular Technology 67 (1) (2018) 44–55. doi:10.1109/TVT.2017.2760281.

[12] H. Peng, X. Shen, Multi-agent reinforcement learning based resource management in mec- and uav-assisted vehicular networks, IEEE Journal on Selected Areas in Communications 39 (1) (2021) 131–141. doi:10.1109/jsac.2020.3036962.
URL http://dx.doi.org/10.1109/jsac.2020.3036962

[13] L. Liang, H. Ye, G. Y. Li, Spectrum sharing in vehicular networks based on multi-agent reinforcement learning, IEEE Journal on Selected Areas in Communications 37 (10) (2019) 2282–2292. doi:10.1109/jsac.2019.2933962.
URL http://dx.doi.org/10.1109/jsac.2019.2933962

[14] X. Wang, Y. Han, V. C. M. Leung, D. Niyato, X. Yan, X. Chen, Convergence of edge computing and deep learning: A comprehensive survey, IEEE Communications Surveys & Tutorials 22 (2) (2020) 869–904. doi:10.1109/comst.2020.2970550.
URL http://dx.doi.org/10.1109/comst.2020.2970550

[15] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, J. Zhang, Edge intelligence: Paving the last mile of artificial intelligence with edge computing, Proceedings of the IEEE 107 (8) (2019) 1738–1762. doi:10.1109/jproc.2019.2918951.
URL http://dx.doi.org/10.1109/jproc.2019.2918951

[16] J. Chen, X. Ran, Deep learning with edge computing: A review, Proceedings of the IEEE 107 (8) (2019) 1655–1674. doi:10.1109/jproc.2019.2921977.
URL http://dx.doi.org/10.1109/jproc.2019.2921977

[17] S. Wang, T. Tuor, T. Salonidis, K. K. Leung, C. Makaya, T. He, K. Chan, Adaptive federated learning in resource constrained edge computing systems, IEEE Journal on Selected Areas in Communications 37 (6) (2019) 1205–1221. doi:10.1109/jsac.2019.2904348.
URL http://dx.doi.org/10.1109/jsac.2019.2904348

[18] W. Y. B. Lim, N. C. Luong, D. T. Hoang, Y. Jiao, Y.-C. Liang, Q. Yang, D. Niyato, C. Miao, Federated learning in mobile edge networks: A comprehensive survey, IEEE Communications Surveys & Tutorials 22 (3) (2020) 2031–2063. doi:10.1109/comst.2020.2986024.
URL http://dx.doi.org/10.1109/comst.2020.2986024

[19] X. Wang, Y. Han, C. Wang, Q. Zhao, X. Chen, M. Chen, In-edge ai: Intelligentizing mobile edge computing, caching and communication by federated learning, IEEE Network 33 (5) (2019) 156–165. doi:10.1109/mnet.2019.1800286.
URL http://dx.doi.org/10.1109/mnet.2019.1800286

[20] Z. Yang, K. Yang, L. Lei, K. Zheng, V. C. M. Leung, Blockchain-based decentralized trust management in vehicular networks, IEEE Internet of Things Journal 6 (2) (2019) 1495–1505. doi:10.1109/jiot.2018.2836144.
URL http://dx.doi.org/10.1109/jiot.2018.2836144

[21] J. Kang, R. Yu, X. Huang, M. Wu, S. Maharjan, S. Xie, Y. Zhang, Blockchain for secure and efficient data sharing in vehicular edge computing and networks, IEEE Internet of Things Journal 6 (3) (2019) 4660–4670. doi:10.1109/jiot.2018.2875542.
URL http://dx.doi.org/10.1109/jiot.2018.2875542

[22] Z. Lu, G. Qu, Z. Liu, A survey on recent advances in vehicular network security, trust, and privacy, IEEE Transactions on Intelligent Transportation Systems 20 (2) (2019) 760–776. doi:10.1109/tits.2018.2818888.
URL http://dx.doi.org/10.1109/tits.2018.2818888

[23] L. Liu, C. Chen, Q. Pei, S. Maharjan, Y. Zhang, Vehicular edge computing and networking: A survey, Mobile Networks and Applications 26 (3) (2020) 1145–1168. doi:10.1007/s11036-020-01624-1.
URL http://dx.doi.org/10.1007/s11036-020-01624-1

[24] L. Huang, S. Bi, Y.-J. A. Zhang, Deep reinforcement learning for online computation offloading in wireless powered mobile-edge computing networks, IEEE Transactions on Mobile Computing 19 (11) (2020) 2581–2593. doi:10.1109/tmc.2019.2928811.
URL http://dx.doi.org/10.1109/tmc.2019.2928811

[25] S. Feng, Y. Zhang, S. E. Li, Z. Cao, H. X. Liu, L. Li, String stability for vehicular platoon control: Definitions and analysis methods, Annual Reviews in Control 47 (2019) 81–97. doi:10.1016/j.arcontrol.2019.03.001.
URL http://dx.doi.org/10.1016/j.arcontrol.2019.03.001

[26] F. Tang, Y. Kawamoto, N. Kato, J. Liu, Future intelligent and secure vehicular network toward 6g: Machine-learning approaches, Proceedings of the IEEE 108 (2) (2020) 292–307. doi:10.1109/jproc.2019.2954595.
URL http://dx.doi.org/10.1109/jproc.2019.2954595

[27] L. Breslau, P. Cao, L. Fan, G. Phillips, S. Shenker, Web caching and Zipf-like distributions: evidence and implications, in: Proceedings of the IEEE INFOCOM '99. Eighteenth Annual Joint Conference of the IEEE Computer and Communications Societies, IEEE, 1999, pp. 126–134. doi:10.1109/INFCOM.1999.749260.